BÁO CÁO BÀI TẬP LẬP TRÌNH MẠNG

Họ tên: Hoàng Tuấn Minh

MSSV: 20235376

Mã lớp: 168503

Link bài tập: hoangtminh/Lap-trinh-mang

1. Bài tập pubsub_server

Lập trình ứng dụng server thực hiện chức năng định tuyến dữ liệu đơn giản như sau:

+ Server chờ các kết nối ở cổng 9000.
+ Client gửi “SUB <topic>” để đăng ký nhận tin nhắn thuộc chủ đề topic.
+ Client gửi “UNSUB <topic>” để hủy đăng ký nhận tin nhắn thuộc chủ đề topic.
+ Client gửi “PUB <topic> <msg>” để gửi tin nhắn msg đến chủ đề topic.
+ Server nhận msg và chuyển tiếp cho tất cả các client đã đăng ký topic trước đó.

Chú ý: Một client có thể đăng ký cùng một lúc nhiều chủ đề khác nhau.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <sys/select.h>

#define MAX_CLIENTS 100
#define MAX_TOPICS 20
#define PORT 9000

typedef struct {
    int fd;
    char topics[MAX_TOPICS][50];
    int num_topics;
} Client;

Client clients[MAX_CLIENTS];
int nClients = 0;

int find_topic_index(int client_idx, char *topic) {
    for (int i = 0; i < clients[client_idx].num_topics; i++) {
        if (strcmp(clients[client_idx].topics[i], topic) == 0) return i;
    }
    return -1;
}

int main() {
    int listener = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(PORT);
```

Code pubsub_server 1

```c
    if (bind(listener, (struct sockaddr *)&addr, sizeof(addr)) == -1) {
        perror("bind() failed");
        return 1;
    }
    listen(listener, 10);
    printf("Pub/Sub Server dang chay tren port %d...\n", PORT);

    fd_set fdread;

    while (1) {
        FD_ZERO(&fdread);
        FD_SET(listener, &fdread);
        int max_fd = listener;

        for (int i = 0; i < nClients; i++) {
            FD_SET(clients[i].fd, &fdread);
            if (clients[i].fd > max_fd) max_fd = clients[i].fd;
        }

        select(max_fd + 1, &fdread, NULL, NULL, NULL);

        if (FD_ISSET(listener, &fdread)) {
            int client_fd = accept(listener, NULL, NULL);
            if (nClients < MAX_CLIENTS) {
                clients[nClients].fd = client_fd;
                clients[nClients].num_topics = 0;
                nClients++;
                printf("Client moi ket noi (FD: %d). Tong: %d\n", client_fd, nClients);
                send(client_fd, "Connected to Pub/Sub Server. Use SUB, UNSUB, PUB.\n", 51,
            } else {
                close(client_fd);
            }
        }

```

Code pubsub_server 2

```c
        for (int i = 0; i < nClients; i++) {
            if (FD_ISSET(clients[i].fd, &fdread)) {
                char buf[1024];
                int ret = recv(clients[i].fd, buf, sizeof(buf) - 1, 0);

                if (ret <= 0) {
                    printf("Client FD %d ngat ket noi\n", clients[i].fd);
                    close(clients[i].fd);
                    clients[i] = clients[nClients - 1];
                    nClients--; i--; continue;
                }

                buf[ret] = '\0';
                while(ret > 0 && (buf[ret-1] == '\n' || buf[ret-1] == '\r')) buf[--ret] = '\0';

                char cmd[10], topic[50];
                int n = sscanf(buf, "%s %s", cmd, topic);

                if (n >= 2) {
                    if (strcmp(cmd, "SUB") == 0) {
                        if (find_topic_index(i, topic) == -1) {
                            if (clients[i].num_topics < MAX_TOPICS) {
                                strcpy(clients[i].topics[clients[i].num_topics++], topic);
                                send(clients[i].fd, "SUB OK\n", 7, 0);
                            } else {
                                send(clients[i].fd, "SUB ERROR: Topic limit reached\n", 31, 0);
                            }
                        } else {
                            send(clients[i].fd, "SUB ERROR: Already subscribed\n", 30, 0);
                        }
                    }
                    else if (strcmp(cmd, "UNSUB") == 0) {
                        int idx = find_topic_index(i, topic);
                        if (idx != -1) {
```

Code pubsub_server 3

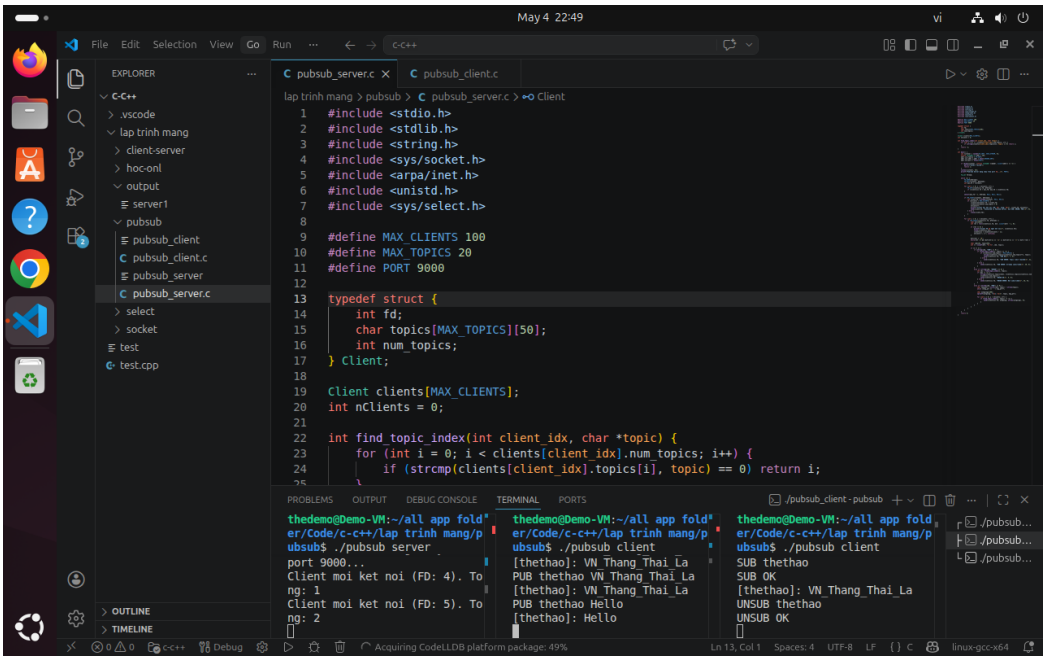
```c
                    if (idx != -1) {
                        strcpy(clients[i].topics[idx], clients[i].topics[clients[i].num_topics
                        clients[i].num_topics--;
                        send(clients[i].fd, "UNSUB OK\n", 9, 0);
                    } else {
                        send(clients[i].fd, "UNSUB ERROR: Not subscribed\n", 28, 0);
                    }
                }
                else if (strcmp(cmd, "PUB") == 0) {
                    char *msg_ptr = strstr(buf, topic) + strlen(topic);
                    while (*msg_ptr == ' ') msg_ptr++;

                    char outgoing[1100];
                    sprintf(outgoing, "[%s]: %s\n", topic, msg_ptr);

                    for (int j = 0; j < nClients; j++) {
                        if (find_topic_index(j, topic) != -1) {
                            send(clients[j].fd, outgoing, strlen(outgoing), 0);
                        }
                    }
                }
            }
        }
    }
    return 0;
}
```

Code pubsub_server 4

- **Kết quả:**



Kết quả pubsub_server